PA4

20225087

성준호

1. Materials / material.h, material.cpp

기본적으로 재질은 다음과 같은 기본적인 렌더링 방정식에 대한 몬테카를로 적분을 사용하여 구현하였다.

$$L_r(\mathrm{p}, \omega_r) = \int_{H^2} f_r(\mathrm{p}, \omega_i \to \omega_r)\, L_i(\mathrm{p}, \omega_i)\, \cos\theta_i \, \mathrm{d}\omega_i$$

$$\frac{1}{N} \sum_{j=1}^{N} \frac{f_r(\mathrm{p}, \omega_j \to \omega_r)\, L_i(\mathrm{p}, \omega_j)\, \cos\theta_j}{p(\omega_j)}$$

따라서 재질을 구현하기 위해서 해당 렌더링 방정식의 재질마다의 고유한 f term (BRDF / BTDF)과 몬테카를로 적분 시 필요한 확률 밀도 함수 PDF가 필요했고, 따라서 해당 함수들을 통해 값을 도출하는 방법을 코드로 표현하는 것이 재질을 구현하는데 필요한 가장 중요한 요소였다. 또한 Out Ray를 생성할 때 BSDF(BRDF + BTDF) 분포에 따라 Out Ray의 방향을 결정하는 방식까지 구현했어야 했다.

먼저 재질에 대한 구현에서 공통적으로 사용되는 함수들은 다음과 같은 역할을 한다.

```
bool scatter(const ne::Ray &r_in, const ne::Intersection &hit,
        ne::Ray &r_out) const override;
```

: 물체에 ray가 충돌했을 때 해당 재질의 BSDF 분포를 기반으로 Out Ray를 Sampling한다.

```
glm::vec3 attenuation(const ne::Ray& r_in, const ne::Intersection& hit, const ne::Ray& r_out, bool is_scatterd) const override;
```

: 물체 고유의 $BSDF * cos(\theta_o)$ 값을 return한다. 렌더링 방정식의 f term과 cos term 계산을 위해 사용된다.

```
float pdf(const ne::Ray& r_in, const ne::Intersection& hit, const ne::Ray& r_out, bool is_scatterd) const override;
```

: 물체에 충돌한 이후의 Ray가 Sampling되는 방식에 따른 PDF 값을 return한다. 렌더링 방정식의 $p(w_i)$ term 계산을 위해 사용된다.

```
// Transfrom world coord to shading coord.
glm::mat3 changeToLocal(glm::vec3 normal) {
    glm::vec3 up = std::abs(normal.y) < 0.999f ? glm::vec3(0, 1, 0) : glm::vec3(1, 0, 0);
    glm::vec3 t = glm::normalize(glm::cross(up, normal));
    glm::vec3 b = glm::cross(normal, t);
    glm::mat3 TBN(t, b, normal);

    return glm::transpose(TBN);
}
```

: 추가로 물체와 Ray의 충돌 지점의 법선 벡터를 Normal로 삼는 로컬 좌표계(z축이 Up)

에서 월드 좌표계로의 변환 함수를 정의했는데, 해당 과제를 구현하면서 참고했던 대부분의 자료에서 Shading 지점의 로컬 좌표계를 기준으로 구현되어 있었기에, 편의를 위해 해당 함수를 따로 정의했다.

이제 각 재질에 대한 BSDF 및 PDF의 수식을 정하고 해당 수식을 코드로 표현하면 재질에 대한 표현이 가능해진다. 해당 수식들의 대부분은 Physically Based Rendering: From Theory to Implementation 4th edition (https://www.pbr-book.org/4ed/contents) (편의상 PBR이라 칭하겠다.)을 참고하여 정의하였다.

1-1. Lambertian

과제 가이드에서 제공된 참고자료에 따르면 Lambertian 물체의 색상은 다음과 같이 표현된다고 한다.

$$\text{Diffuse Surface Color} = \frac{\rho_d}{\pi} \cdot L_i \cdot \cos\theta.$$

이를 통해 Lambertian 물체의 BRDF가 $\frac{color}{\pi}$ 라는 상수 값이라는 것을 알 수 있었고, 따라서 Lambertian의 attenuation() 함수는 다음과 같이 정의하였다.

```
glm::vec3 Lambertian::attenuation(const ne::Ray& r_in, const ne::Intersection& hit, const ne::Ray& r_out, bool is_scatterd) const {
    return color_ / glm::pi<float>() * glm::dot(hit.n, r_out.dir);
}
```

glm::dot(hit.n, r_out.dir)은 $cos(\theta_o)$ term이다.

Lambertian 물체의 attenuation의 정의는 완료되었으므로, Ray가 물체에 충돌한 이후에 반사될 방향을 결정해주는 함수인 scatter()를 정의하고자 하였다. 해당 방식에는 PBR에 따르면 2개가 있었는데, Uniform Hemisphere Sampling과 Cosine Weighted Hemisphere Sampling이었다. 그 중 PBR에는 Cosine Weighted Hemisphere Sampling이 Lambertian 물체에 효과적이라고 나타나 있으며, 따라서 Cosine Weighted Hemisphere Sampling으로 scatter() 함수를 구현하고자 하였다. PBR 4ed A.5.3에서 설명하는 Cosine Weighted Hemisphere Sampling의 방식은 다음과 같다.

1. Concentric Disk Mapping을 통해 충돌 지점의 법선 벡터를 기준으로 하는 단위 원판 위에 Random한 Point를 선택한다. 해당 방식은 원판 위의 한 점을 무작위로 선택할 때 기존의 단순한 Polar coordinate 기반의 선택보다 균일한 무작위성을 보장한다. (PBR 4ed A.5.1)

2. 선택한 지점을 단위 반구로 투영한다.

3. 충돌 지점에서 투영된 지점으로 Ray를 Sample한다.

이 방식을 코드로 표현한 결과가 다음과 같았다.

```
bool Lambertian::scatter(const ne::Ray& r_in, const ne::Intersection& hit,
    ne::Ray& r_out) const {

    // Sample Disk Uniform Concentric.
    float u1 = glm::linearRand(0.0f, 1.0f);
    float u2 = glm::linearRand(0.0f, 1.0f);

    float uOffsetx = 2 * u1 - 1;
    float uOffsety = 2 * u2 - 1;

    float r, theta;

    if (uOffsetx == 0 && uOffsety == 0) {
        r = 0;
        theta = 0;
    }
    else if (std::abs(uOffsetx) > std::abs(uOffsety)) {
        r = uOffsetx;
        theta = (glm::pi<float>() / 4) * (uOffsety / uOffsetx);
    }
    else {
        r = uOffsety;
        theta = (glm::pi<float>() / 2) - (glm::pi<float>() / 4)* (uOffsetx / uOffsety);
    }

    // Project random sample points on uniform disk up to the unit hemisphere.
    float x = r * cos(theta);
    float y = r * sin(theta);
    float z = sqrt(std::max(0.0f, 1.0f - x * x - y * y));

    // Change to world coordinate system.
    glm::vec3 rand_dir(x, y, z);
    rand_dir = glm::transpose(changeToLocal(hit.n))* rand_dir;

    r_out.dir = glm::normalize(rand_dir);
    r_out.o = hit.p + 1e-4f * r_out.dir;

    return true;
}
```

Shading 지점의 법선 벡터를 기준으로 하여, 법선을 z축(Up)으로 두는 로컬 좌표계에서 Random Sampling을 진행하고 changeToLocal(hit.n) 함수로부터 얻은 matrix의 transpose를 곱해주어 월드 좌표계로 변환한다.

Because directional PDFs in `pbrt` are defined with respect to solid angle, the PDF function returns the value $\cos\theta/\pi$.

PBR에 따르면 Cosine Weighted Hemisphere Sampling을 진행하고, 입체각에 대해 PDF

를 정의할 시 Lambertian 물체의 PDF는 $cos(\theta_i)$ / π로 계산된다고 한다. (PBR 4ed A.5.3) 따라서 pdf() 함수를 다음과 같이 정의한다.

```
float Lambertian::pdf(const ne::Ray& r_in, const ne::Intersection& hit, const ne::Ray& r_out, bool is_scatterd) const {
    return glm::dot(hit.n, r_out.dir) / glm::pi<float>();
}
```

PDF를 attenuation()에서 같이 계산하지 않는 이유는 추후 Direct Light Sampling에서 Direct Light에 대해서는 다른 PDF를 사용해야 하기 때문에 따로 분리하였다.

Integrator.cpp의 integrate 함수를 다음과 같이 수정한 후 렌더링을 진행한 결과는 다음과 같다. (주석 처리된 부분은 Direct Light Sample과 관련된 부분으로 이후에 다룬다.)

```
// if ray hit object in scene, apply shading
if (scene->rayIntersect(r, hit)) {
    RendablePointer object = hit.object;
    MaterialPointer material = hit.material;

    // avoid double counting
    //if (std::find(dls_objects.begin(), dls_objects.end(), object) == dls_objects.end())
        L += material->emitted();

    ne::Ray out_ray;
    if (hit.material->scatter(r, hit, out_ray)) {

        // Direct Light Sampling
        //dls_objects.clear();
        //glm::vec3 d_L = scene->sampleDirectLight(r, hit, dls_objects);
        //if (d_L != glm::vec3(0.0f))
        //    L += d_L;

        const float pdf = material->pdf(r, hit, out_ray, true);
        if (pdf < 1e-6f)
            return L;
        const glm::vec3 albedo = material->attenuation(r, hit, out_ray, true);
        //L += albedo * scene->sampleBackgroundLight(out_ray.dir) / pdf;
        L += albedo * this->integrate(out_ray, scene, ray_depth - 1, dls_objects) / pdf;
    }
}
```

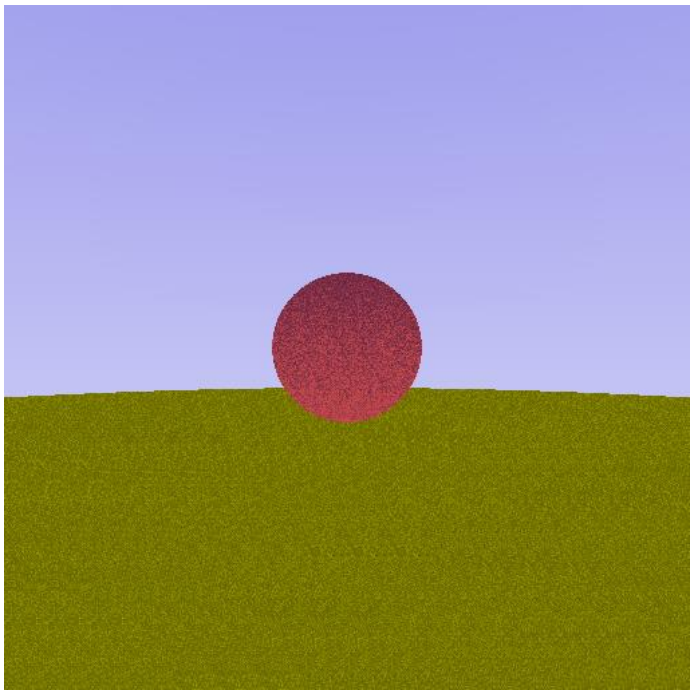


Fig1. Lambertian_512x512_spp1

1-2. Dielectric

Dielectric의 경우 Glass와 같은 재질이며, 해당 재질은 반사만 일어나는 Lambertian과 같은 Diffuse 물체와 달리 반사와 투과 모두 일어나는 재질이다. 따라서 반사의 분포만을 고려하는 BRDF(Bidirectional reflectance distribution function)뿐만 아니라 투과의 분포를 고려하는 BTDF(Bidirectional transmittance distribution function)를 합친 BSDF(Bidirectional scattering distribution function)를 구해줘야 했다.

먼저 Dielectric의 scatter() 함수부터 정의해보자. 과제에서는 Perfect Glass만을 상정하고 있기에, Dielectric을 Perfect Glass와 같은 Specular Material로 간주하겠다. PBR 4ed 9.3.5에 따르면 Perfect Glass와 같은 Specualr Material에선 입사 광량의 일부는 정반사되고 일부는 투과된다고 하며, 그리고 그 비율은 Fresnel 방정식을 통해 구할 수 있다고 한다.

$$r_{\parallel} = \frac{E_{\mathrm{r}}^{\parallel}}{E_{\mathrm{i}}^{\parallel}} = \frac{\eta_{\mathrm{t}} \cos\theta_{\mathrm{i}} - \eta_{\mathrm{i}} \cos\theta_{\mathrm{t}}}{\eta_{\mathrm{t}} \cos\theta_{\mathrm{i}} + \eta_{\mathrm{i}} \cos\theta_{\mathrm{t}}},$$

$$r_{\perp} = \frac{E_{\mathrm{r}}^{\perp}}{E_{\mathrm{i}}^{\perp}} = \frac{\eta_{\mathrm{i}} \cos\theta_{\mathrm{i}} - \eta_{\mathrm{t}} \cos\theta_{\mathrm{t}}}{\eta_{\mathrm{i}} \cos\theta_{\mathrm{i}} + \eta_{\mathrm{t}} \cos\theta_{\mathrm{t}}}.$$

$$F_{\mathrm{r}} = \frac{1}{2}\left(r_{\parallel}^{2} + r_{\perp}^{2}\right)$$

위 식에서 Fr이 Fresnel 계수 중 반사율에 대한 값으로, 정반사되는 빛의 비율을 나타낸다. Specular Material에선 빛을 흡수하지 않기 때문에, 곧 $1 - Fr$이 굴절되어 투과되는 빛의 비율임을 알 수 있다. n값의 경우 재질 마다의 굴절률(neon sandbox에선 IOR로 제공)로 진공 상태의 굴절률이 기본 값인 1.0이다. 따라서 scatter() 함수는 Fresnel 반사 계수를 구하고 [0, 1]에 해당되는 Random한 값을 추출하여 반사 계수보다 낮을 시 반사되고, 높을 시 굴절되어 투과되는 방식으로 구현하면 될 것임을 알 수 있었다. 만약 Fr 값이 1과 같다면 이는 모든 빛이 반사되는 전반사 상황이다.

다음은 이를 구현한 코드이다.

```
bool Dielectric::scatter(const ne::Ray &r_in, const ne::Intersection &hit,
                         ne::Ray &r_out) const {

    // Calculate Fresnel Equation.
    glm::vec3 normal = glm::normalize(hit.n);
    float cosTheta_i = glm::dot(-r_in.dir, normal);
    cosTheta_i = glm::clamp(cosTheta_i, -1.0f, 1.0f);

    float IOR_in = 1.0f;
    float IOR_out = this->IOR_;

    if (cosTheta_i < 0.0f) {
        std::swap(IOR_in, IOR_out);
        cosTheta_i = -cosTheta_i;
        normal = -normal;
    }

    float eta = IOR_in / IOR_out;

    float sin2Theta_i = 1.0f - cosTheta_i * cosTheta_i;
    float sin2Theta_t = sin2Theta_i * (eta * eta);
    // Total Reflection
    if (sin2Theta_t >= 1.0f) {
        r_out.dir = glm::normalize(glm::reflect(r_in.dir, normal));
        r_out.o = hit.p + 1e-4f * r_out.dir;
        return true;
    }
    float cosTheta_t = std::sqrt(std::max(0.0f, 1.0f - sin2Theta_t));

    float r_s = (IOR_in * cosTheta_i - IOR_out * cosTheta_t) / (IOR_in * cosTheta_i + IOR_out * cosTheta_t);
    float r_p = (IOR_out * cosTheta_i - IOR_in * cosTheta_t) / (IOR_out * cosTheta_i + IOR_in * cosTheta_t);

    float R = (r_s * r_s + r_p * r_p) / 2.0f;

    // Reflection
    if (glm::linearRand(0.0f, 1.0f) <= R) {
        r_out.dir = glm::normalize(glm::reflect(r_in.dir, normal));
        r_out.o = hit.p + 1e-4f * r_out.dir;
    }
    // Transmission
    else {
        r_out.dir = glm::normalize(glm::refract(r_in.dir, normal, eta));
        r_out.o = hit.p + 1e-4f * r_out.dir;
    }
    return true;
```

Neon sandbox에서는 구의 내부에서 빛이 충돌하든 외부에서 충돌하든 법선 벡터는 구의 외부 표면에 대해 생성된다. 따라서 입사광(충돌 지점으로 들어오는 방향이기에 뒤집어준다.)과 충돌 지점의 법선 벡터의 코사인 값이 0보다 작다면 이는 곧 빛이 유리 구 안에서 밖으로 나가는 상황이기에, 빛이 들어오는 매질의 굴절률과 나가는 매질의 굴절률을 바꿔주고, 법선 벡터를 뒤집어서 입사광을 기준으로 바꿔준 후 코사인 값에도 -1을 곱해주어 빛이 구 내부에서 외부로 나가는 상황으로 변경시켜준다. 반사와 굴절에 대한 계산(굴절 시 Snell's law 계산 등)은 glm에서 제공하는 reflect() 및 refract() 함수를 사용하여 따로 코드로 구현하지 않아도 가능했다.

attenuation() 함수의 경우 구현이 간단했다. scatter() 함수를 보면 빛이 Specular Material에 충돌한다면 정반사 되거나 굴절되어 투과되는 단 2가지의 경로 밖에 존재하지 않는다는 것을 알 수 있다. 따라서 Diffuse Material의 attenuation()과는 다르게 입사

광과 반사광의 방향을 고려해주지 않아도 됨을 알 수 있다. PBR 3ed 8.2.2를 참고하면 반사 시의 BRDF는 다음과 같다고 한다.

$$f_{\mathrm{r}}(\mathrm{p}, \omega_{\mathrm{o}}, \omega_{\mathrm{i}}) = F_{\mathrm{r}}(\omega_{\mathrm{r}}) \frac{\delta(\omega_{\mathrm{i}} - \omega_{\mathrm{r}})}{|\cos\theta_{\mathrm{r}}|}$$

코사인 term은 렌더링 방정식의 코사인 term을 상쇄시키기 위함이고, δ term의 경우 디렉 델타 함수로 반사광의 방향이 정확히 정반사의 방향이 아닐 경우 BRDF를 0으로 계산하기 위한 term이다. 즉, BRDF를 렌더링 방정식에 적용시켰을 때 결과가 다음과 같이 계산되도록 설정한 BRDF임을 알 수 있다.

$$L_{\mathrm{o}}(\omega_{\mathrm{o}}) = \int f_{\mathrm{r}}(\omega_{\mathrm{o}}, \omega_{\mathrm{i}})\, L_{\mathrm{i}}(\omega_{\mathrm{i}})\, |\cos\theta_i|\, \mathrm{d}\omega_{\mathrm{i}} = F_{\mathrm{r}}(\omega_{\mathrm{r}})\, L_{\mathrm{i}}(\omega_{\mathrm{r}})$$

이는 Specular Material에 들어온 빛의 경우 오로지 한 방향에만 영향을 미친다는 사실을 반영한 결과라고 볼 수 있다. 따라서 최종적으로 attenuation()은 Fresnel 반사 계수만 계산하면 될 것임을 알 수 있었다.

$$f_{\mathrm{r}}(\omega_{\mathrm{o}}, \omega_{\mathrm{i}}) = \frac{\eta_{\mathrm{o}}^2}{\eta_{\mathrm{i}}^2}(1 - F_{\mathrm{r}}(\omega_{\mathrm{i}})) \frac{\delta(\omega_{\mathrm{i}} - \mathrm{T}(\omega_{\mathrm{o}}, \mathbf{n}))}{|\cos\theta_{\mathrm{i}}|}$$

투과시의 BTDF의 경우 다음과 같았음을 알 수 있는데, BRDF와 달리 바깥 매질과 안쪽 매질의 굴절률의 비율에 대한 제곱 값이 추가로 곱해진 것을 알 수 있다. PBR 3ed 8.2.3에 따르면 이는 빛이 굴절시에 방사 휘도(radiance)가 변하기 때문으로 굴절시에 입사광과 굴절광의 법선에 대한 각도 차이로 인해 발생하는 변화량을 보존해주기 위한 term임을 알 수 있다. 다만 공기에서 유리, 다시 유리 내부에서 공기로 이어지는 빛의 흐름에서는 굴절이 두 번 일어나 결국은 곱해져서 상쇄되는 값으로 공기에서 시작된 빛이 유리를 통해 다시 공기로 나오는 경우 해당 빛의 광량은 동일하다는 것을 알 수 있다. 과제에서는 오로지 Sphere만 존재하는 공간을 상정하기 때문에, 유리 구 내부로 들어간 빛은 반드시 유리를 거쳐 다시 공기로 나오게 되어있다. 결국 해당 term은 반드시 상쇄되는 상황이 나오기 때문에 attenuation() 함수에서는 굳이 이를 반영해줄 필요가 없다고 판단하였다.

PDF의 경우 자명한데 반사일 경우 $\frac{Fr}{\{Fr + (1 - Fr)\}} = Fr$, 투과일 경우 $1 - Fr$이다. 결국 정반사 아니면 굴절만 일어나기 때문이다. 따라서 $\frac{BSDF * cos(\theta_o)}{PDF}$의 경우 반사 시에나 굴절

시에나 똑같이 1임을 알 수 있다. Dielectric의 경우 scatter상황을 제외하고는 attenuation()과 pdf()를 따로 사용할 일이 없기 때문에 $\frac{BSDF * cos(\theta_o)}{PDF}$를 미리 계산해서 반환하도록 구현한다.

```
glm::vec3 Dielectric::attenuation(const ne::Ray& r_in, const ne::Intersection& hit, const ne::Ray& r_out, bool is_scatterd) const {
    if (!is_scatterd)
        return glm::vec3(0.0f);

    return glm::vec3(1.0f);
}

float Dielectric::pdf(const ne::Ray& r_in, const ne::Intersection& hit, const ne::Ray& r_out, bool is_scatterd) const {
    if (!is_scatterd)
        return 0.0f;

    return 1.0f;
}
```

구현 시에 매개 변수인 is_scattered를 BSDF의 디렉 델타 함수를 표현해주기 위해 사용하였다. 즉, Ray가 scatter() 함수로 계산된 상황이 아니라면 BSDF, PDF의 값을 0으로 반환하기 위함이다.

결과는 다음과 같다.

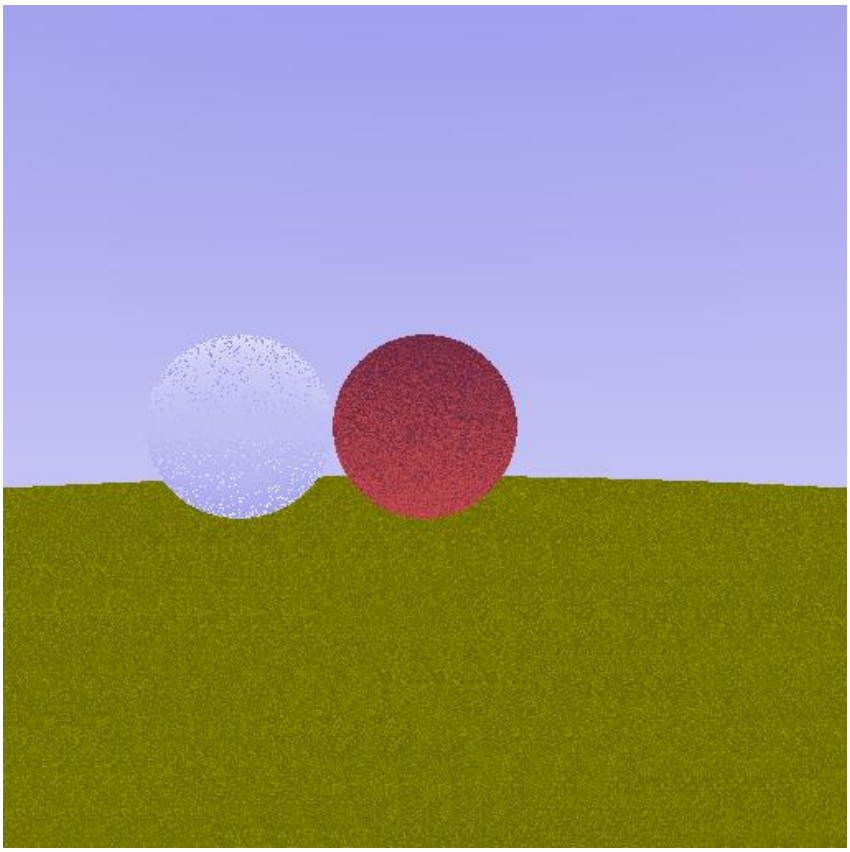


Fig2. Dielectric_512x512_spp1

1-3. Metal

PBR 4ed 9.6을 참고하면 Metal과 같은 재질은 Microfacet Theory를 따르는 재질이다. Mircofacet Theory란 재질의 표면이 Specular Material과 같이 정반사를 일으키는 아주 작고 미세한 Microfacet으로 이루어져 있다고 보는 이론으로, 기본적으로 매끈한(roughness ≅ 0) 재질 표면에서는 Specular Material과 같이 정반사가 일어나지만, 그 표면이 매끈하지 않고 거칠 경우에는 정반사를 일으키는 Microfacet이 기울기를 지닌 채 분포 되어있어 반사되는 빛의 방향이 분산된다고 보는 이론이다. Microfacet Theory를 표현하기 위해서는 아래의 그림과 같은 세 가지 요소가 중요하다고 설명된다.

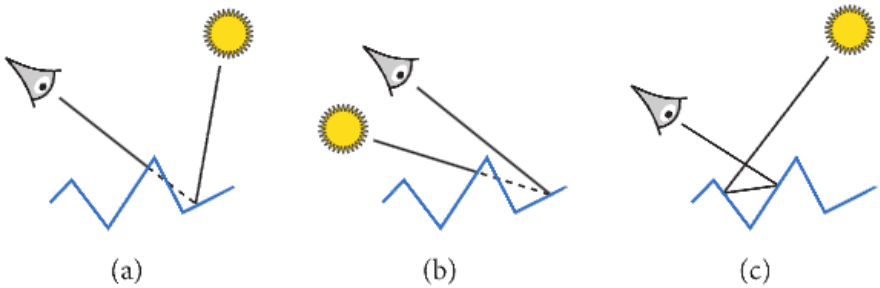


Figure 9.21: Three Important Geometric Effects to Consider with Microfacet Reflection Models. (a) *Masking*: the microfacet of interest is not visible to the viewer due to occlusion by another microfacet. (b) *Shadowing*: analogously, light does not reach the microfacet. (c) *Interreflection*: light bounces among the microfacets before reaching the viewer.

PBR 4th ed Fig 9.21

PBR 4ed 9.6.5을 참고하면 이 세 가지 요소를 고려한 BRDF, PDF, Sampling 방식으로 Torrance–Sparrow Model이 있다는 것을 알 수 있고, 따라서 Metal을 구현하기 위해 Torrance–Sparrow Model이 제공하는 BRDF, PDF, Sampling 방식을 사용하고자 하였다.

먼저 BRDF는 다음과 같다. (PBR 4ed 9.6.5)

$$f_{\mathrm{r}}(\mathrm{p}, \omega_{\mathrm{o}}, \omega_{\mathrm{i}}) = \frac{D(\omega_{\mathrm{m}})\, F(\omega_{\mathrm{o}} \cdot \omega_{\mathrm{m}})\, G(\omega_{\mathrm{i}}, \omega_{\mathrm{o}})}{4 \cos\theta_{\mathrm{i}}\ \cos\theta_{\mathrm{o}}}$$

D term은 Microfacet들이 ω_{m}(half vector, 입사광과 반사광의 합 벡터를 normalize한 벡터)를 향할 확률을 나타내주는 term으로, 표면 위에 얼마나 많은 Microfacet이 half vector 방향을 향하고 있는지를 나타내주는 함수이다. 수식은 다음과 같다. (PBR 4ed 9.6.1)

$$D(\omega_{\mathrm{m}}) = \frac{1}{\pi\,\alpha_x\,\alpha_y\,\cos^4\theta_{\mathrm{m}}\left(1+\tan^2\theta_{\mathrm{m}}\left(\frac{\cos^2\phi_{\mathrm{m}}}{\alpha_x^2}+\frac{\sin^2\phi_{\mathrm{m}}}{\alpha_y^2}\right)\right)^2}.$$

여기서 α의 값은 roughness에 대한 값으로 neon sandbox에서 Metal에 대한 재질에 기본적으로 주어지는 roughness_ 값이 그것이다. 해당 수식은 비등방성, 즉 거칠기가 x, y 축에 따라 다른 상황을 설명하는 수식으로, neon sandbox에선 roughness 값이 x, y축에 따라 다르지 않고 하나의 상수로 주어지기 때문에 해당 수식에서 α의 값을 x와 y로 따로 나누지 않고 하나의 값으로 계산해야 한다. 이 경우 수식은 다음과 같이 변경된다.

$$D(\omega_m) = \frac{1}{\pi\alpha^2\cos^4\theta_m\left(1+\frac{\tan^2\theta_m}{\alpha^2}\right)^2}$$

결과적으로 코사인 제곱과 사인 제곱의 합은 1이기 때문에 상쇄되고, 알파의 값은 제곱으로 들어가는 방식으로 바뀌게 된다. 수식에서 θ_m 은 half vector의 충돌 지점의 normal vector에 대한 각도이다. α의 값은 roughness의 제곱으로 설정한다.

다음으로 F term이다. F term의 경우 Dielectric에서 사용된 Fresnel 방정식이다. 그런데 Fresnel 방정식의 경우 재질의 굴절률이 주어져야만 계산이 가능한 수식으로, 현재 neon sandbox 환경에서는 Metal에서는 계산하기가 어려운 수식이었다. 따라서 이를 대체할 수 있는 방법을 조사해본 결과 Fresnel 방정식에 대한 근사식으로 Shlick 근사가 존재함을 알게 되었고, 수식은 다음과 같았다. (Wikipedia)

$$R(\theta) = R_0 + (1-R_0)(1-\cos\theta)^5$$

해당 수식에서 또한 R0의 값은 굴절률이 필요하지만, 간단히 Metal의 RGB 값으로 대체 가능하다. 마찬가지로 θ 의 경우에는 half vector와 충돌 지점의 normal vector에 대한 각도이다. 따라서 F term 계산에는 해당 식을 사용한다.

마지막으로 G term이다. G term은 Masking-Shadowing Function으로 Microfacet끼리 가리거나 그림자지는 현상을 보정해주는 함수이다.

$$G(\omega_{\mathrm{o}},\omega_{\mathrm{i}}) = \frac{1}{1+\Lambda(\omega_{\mathrm{o}})+\Lambda(\omega_{\mathrm{i}})}$$

$$\Lambda(\omega) = \frac{\sqrt{1+\alpha^2\tan^2\theta}-1}{2}$$

수식은 위와 같이 비교적 간단했다.

따라서 이들을 종합하여 계산하는 attenuation() 코드는 다음과 같다.

```
glm::vec3 Metal::attenuation(const ne::Ray& r_in, const ne::Intersection& hit, const ne::Ray& r_out, bool is_scatterd) const {

    glm::vec3 wo = changeToLocal(hit.n) * -r_in.dir;
    glm::vec3 wi = changeToLocal(hit.n) * r_out.dir;

    float cosTheta_o = std::abs(wo.z);
    float cosTheta_i = std::abs(wi.z);
    if (cosTheta_i == 0 || cosTheta_o == 0)
        return glm::vec3(0.0f);

    // Get half vector h.
    glm::vec3 h = wo + wi;
    if (h == glm::vec3(0.0f))
        return glm::vec3(0.0f);
    h = glm::normalize(h);

    // Term calculate factors.
    float alpha = this->roughness_ * this->roughness_;;
    float Cos2Theta;
    float Tan2Theta;

    // D term calculate. (Trowbridge Reitz Distribution)
    float D;
    Cos2Theta = h.z * h.z;
    Tan2Theta = std::max(0.0f, 1.0f - Cos2Theta) / Cos2Theta;
    if (std::isinf(Tan2Theta))
        D = 0.0f;
    else {
        float Cos4Theta = Cos2Theta * Cos2Theta;
        float e = Tan2Theta / (alpha * alpha);
        D = 1.0f / (glm::pi<float>() * alpha * alpha * Cos4Theta * (1 + e) * (1 + e));
    }

    // F term calculate. (Schlick Approximation)
    glm::vec3 F;
    glm::vec3 F0 = this->color_;
    float cos = glm::clamp(glm::dot(wo, h), 0.0f, 1.0f);
    F = F0 + (1.0f - F0) * std::powf(1.0f - cos, 5);

    // G term calculate. (Masking Function)
    float G;
    float lambda1, lambda2;
    Cos2Theta = wo.z * wo.z;
    Tan2Theta = std::max(0.0f, 1.0f - Cos2Theta) / Cos2Theta;
    if (std::isinf(Tan2Theta)) lambda1 = 0.0f;
    else lambda1 = (sqrt(1 + alpha * alpha * Tan2Theta) - 1.0f) / 2.0f;
    Cos2Theta = wi.z * wi.z;
    Tan2Theta = std::max(0.0f, 1.0f - Cos2Theta) / Cos2Theta;
    if (std::isinf(Tan2Theta)) lambda2 = 0.0f;
    else lambda2 = (sqrt(1 + alpha * alpha * Tan2Theta) - 1.0f) / 2.0f;
    G = 1.0f / (1.0f + lambda1 + lambda2);

    // BRDF * cos(wi)
    return D * F * G / (4.0f * cosTheta_o);
}
```

분모의 입사각에 대한 코사인 term은 렌더링 방정식의 코사인 term과 상쇄된다.

Sampling 방식은 Sampling the Distribution of Visible Normals 방식을 따른다. (PBR 4ed 9.6.4) 해당 방식은 간단히 설명하면 입사광을 normal로 하고 roughness가 곱해진 타원 (현재 neon sandbox에서 다루는 방식은 등방성이기 때문에 원이다.) 반구의 시야에 보이는 부분에 대한 분포에서 Random Sampling을 진행하는 방식이다. 가시 normal 방향(wh)을 기반으로 좌표계를 구성하고, Random Sampling을 진행한 후에 다시 Shading local 좌표계로 변환시켜주는 방식이다. Neon Sandbox에선 이후에 월드 좌표계로 다시 변환해주는 과정까지 거쳐야 한다. 코드로 표현한 결과는 다음과 같다.

```
bool Metal::scatter(const ne::Ray& r_in, const ne::Intersection& hit,
    ne::Ray& r_out) const {

    // Sampling the Distribution of Visible Normals.
    float alpha = roughness_ * this->roughness_;
    glm::vec3 wo = changeToLocal(hit.n) * -r_in.dir;
    glm::vec3 wh = glm::normalize(glm::vec3(alpha * wo.x, alpha * wo.y, wo.z));
    if (wh.z < 0) wh = -wh;

    glm::vec3 T1 = (wh.z < 0.999f) ? glm::normalize(glm::cross(glm::vec3(0, 0, 1), wh)) : glm::vec3(1, 0, 0);
    glm::vec3 T2 = glm::cross(wh, T1);

    float u1 = glm::linearRand(0.0f, 1.0f);
    float u2 = glm::linearRand(0.0f, 1.0f);

    float r = std::sqrt(u1);
    float theta = 2.0f * glm::pi<float>() * u2;
    float x = r * cos(theta);
    float y = r * sin(theta);

    float h = std::sqrt(1 - x * x);
    float s = (1.0f + wh.z) / 2.0f;
    y = (1.0f - s) * h + s * y;
    float z = std::sqrt(std::max(0.0f, 1.0f - x * x - y * y));

    glm::vec3 nh = x * T1 + y * T2 + z * wh;
    nh = glm::normalize(glm::vec3(alpha * nh.x, alpha * nh.y, std::max(0.0f, nh.z)));

    glm::vec3 wi = glm::reflect(-wo, nh);
    if (wo.z * wi.z <= 0)
        return false;

    r_out.dir = glm::normalize(glm::transpose(changeToLocal(hit.n)) * wi);
    r_out.o = hit.p + 1e-4f * r_out.dir;

    return true;
}
```

해당 방식으로 Sampling을 진행하면 PDF는 다음의 수식으로 표현된다. (PBR 4ed 9.6.5)

$$p(\omega_{\mathrm{i}}) = D_{\omega_{\mathrm{o}}}(\omega_{\mathrm{m}}) \frac{\mathrm{d}\omega_{\mathrm{m}}}{\mathrm{d}\omega_{\mathrm{i}}} = \frac{D_{\omega_{\mathrm{o}}}(\omega_{\mathrm{m}})}{4(\omega_{\mathrm{o}} \cdot \omega_{\mathrm{m}})}$$

BRDF의 D term과 달라 보이지만, BRDF의 본 수식은 다음과 같으며

$$f_{\mathrm{r}}(\mathrm{p}, \omega_{\mathrm{o}}, \omega_{\mathrm{i}}) = \frac{D_{\omega_{\mathrm{o}}}(\omega_{\mathrm{m}})\, F(\omega_{\mathrm{o}} \cdot \omega_{\mathrm{m}})\, G_1(\omega_{\mathrm{i}})}{4\,(\omega_{\mathrm{o}} \cdot \omega_{\mathrm{m}})\,|\cos\theta_{\mathrm{i}}|}$$

해당 수식의 Dwo term은

$$D_{\omega}(\omega_{\mathrm{m}}) = \frac{G_1(\omega)}{\cos\theta} D(\omega_{\mathrm{m}})\ \max(0, \omega \cdot \omega_{\mathrm{m}})$$

와 같아 해당 Dwo 텀을 대입하여 정리한 수식이 앞서 소개했던 BRDF 수식이라고 보면 된다. D term은 순수하게 normal 방향 ωm을 갖는 Microfacet 면적의 밀도만 기술하는 분포이고, Dwo term은 관찰 방향(출사 방향) ωo에서 실제로 보이는(유효한) Microfacet Normal만 모아서 만든 조건부 분포로, 보이는 가시 반구에 대해서만 Sampling하는 분포를 고려하는 분포이기에 Masking function인 G1 term이 추가로 붙는 등의 수식 차이가 존재하는 함수이다.

$$p(\omega_i) = \frac{G_1(\omega_o)}{4\cos\theta_o} D(\omega_m), \quad G_1(\omega) = \frac{1}{1 + \Lambda(\omega)}$$

이는 PDF term과 G1 term을 다시 정리한 결과이고, 따라서 이를 코드로 표현하면 pdf() 함수가 된다.

```
float Metal::pdf(const ne::Ray& r_in, const ne::Intersection& hit, const ne::Ray& r_out, bool is_scatterd) const {

    glm::vec3 wo = changeToLocal(hit.n) * -r_in.dir;
    glm::vec3 wi = changeToLocal(hit.n) * r_out.dir;

    float cosTheta_o = std::abs(wo.z);
    float cosTheta_i = std::abs(wi.z);
    if (cosTheta_i == 0 || cosTheta_o == 0)
        return 0.0f;

    // Get half vector h.
    glm::vec3 h = wo + wi;
    if (h == glm::vec3(0.0f))
        return 0.0f;
    h = glm::normalize(h);

    // Term calculate factors.
    float alpha = this->roughness_ * this->roughness_;;
    float Cos2Theta;
    float Tan2Theta;

    // D term calculate.
    float D;
    Cos2Theta = h.z * h.z;
    Tan2Theta = std::max(0.0f, 1.0f - Cos2Theta) / Cos2Theta;
    if (std::isinf(Tan2Theta))
        D = 0;
    else {
        float Cos4Theta = Cos2Theta * Cos2Theta;
        float e = Tan2Theta / (alpha * alpha);
        D = 1.0f / (glm::pi<float>() * alpha * alpha * Cos4Theta * (1 + e) * (1 + e));
    }
```

```
// G1 term calculate.
float G1;
float lambda1;
Cos2Theta = wo.z * wo.z;
Tan2Theta = std::max(0.0f, 1.0f - Cos2Theta) / Cos2Theta;
if (std::isinf(Tan2Theta)) lambda1 = 0.0f;
else lambda1 = (sqrt(1 + alpha * alpha * Tan2Theta) - 1.0f) / 2.0f;
G1 = 1.0f / (1.0f + lambda1);

return D * G1 * std::max(0.0f, glm::dot(wo,h)) / (4.0f * glm::dot(wo, h) * cosTheta_o);
}
```

종합하면 Metal을 구현할 수 있고 결과는 다음과 같다.

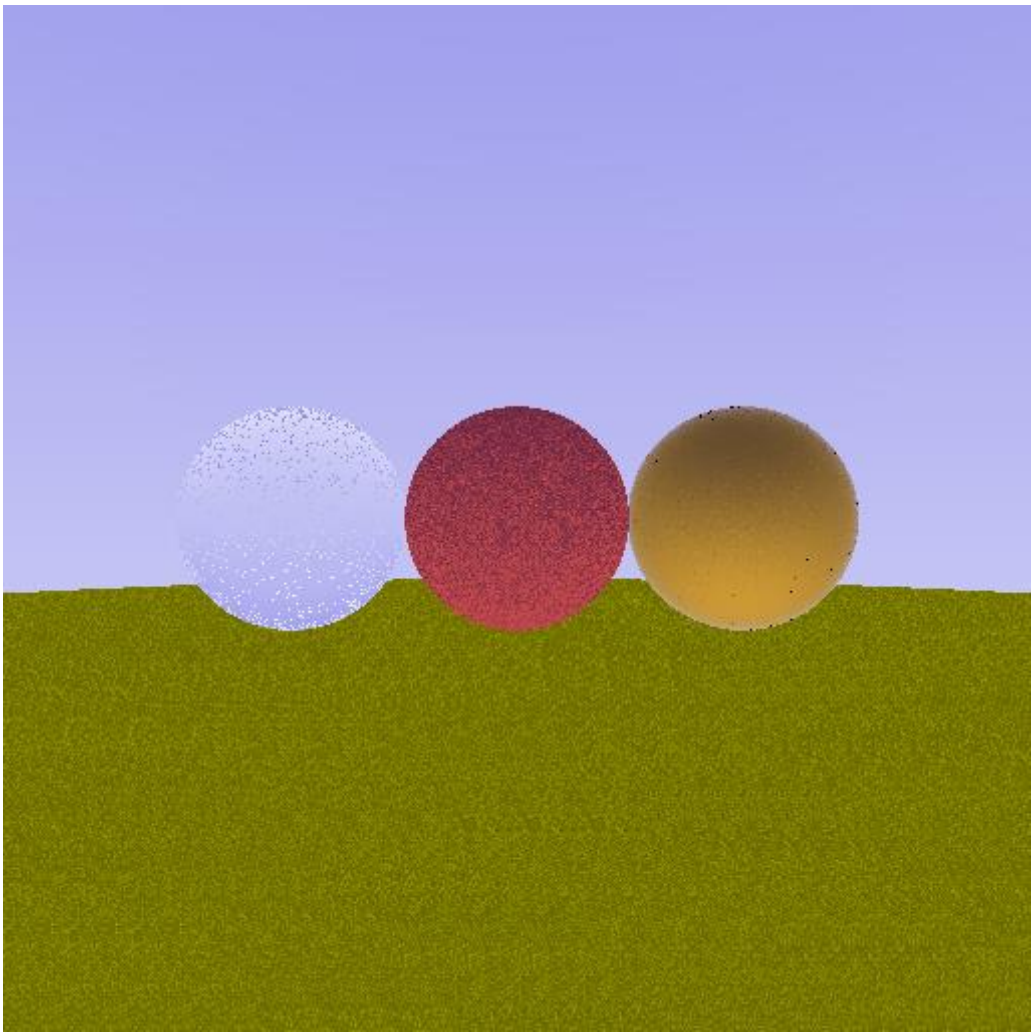


Fig3. Metal_512x512_spp1

1-4. Area Light

Area Light의 경우 굉장히 단순하게 구현이 가능했다. 광원의 경우 빛을 반사하거나 투과시키지 않으므로 scatter() 함수에서 false를 반환하면 되고, 마찬가지로 attenuation() 함수에서도 0 벡터를 반환하면 된다. emitted() 함수의 경우에만 광량(RGB) 벡터를 반환해준다. Light를 제외한 다른 Material에서는 emitted() 함수가 0 벡터를 반환한다.

```
bool DiffuseLight::scatter(const ne::Ray &r_in, const ne::Intersection &hit,
                           ne::Ray &r_out) const {
  return false;
}

glm::vec3 DiffuseLight::emitted() const {
  return this->color_;
}

glm::vec3 DiffuseLight::attenuation(const ne::Ray& r_in, const ne::Intersection& hit, const ne::Ray& r_out, bool is_scatterd) const {
  return glm::vec3(0.0f);
}
```

종합적으로 모든 Material을 구현한 결과이다.

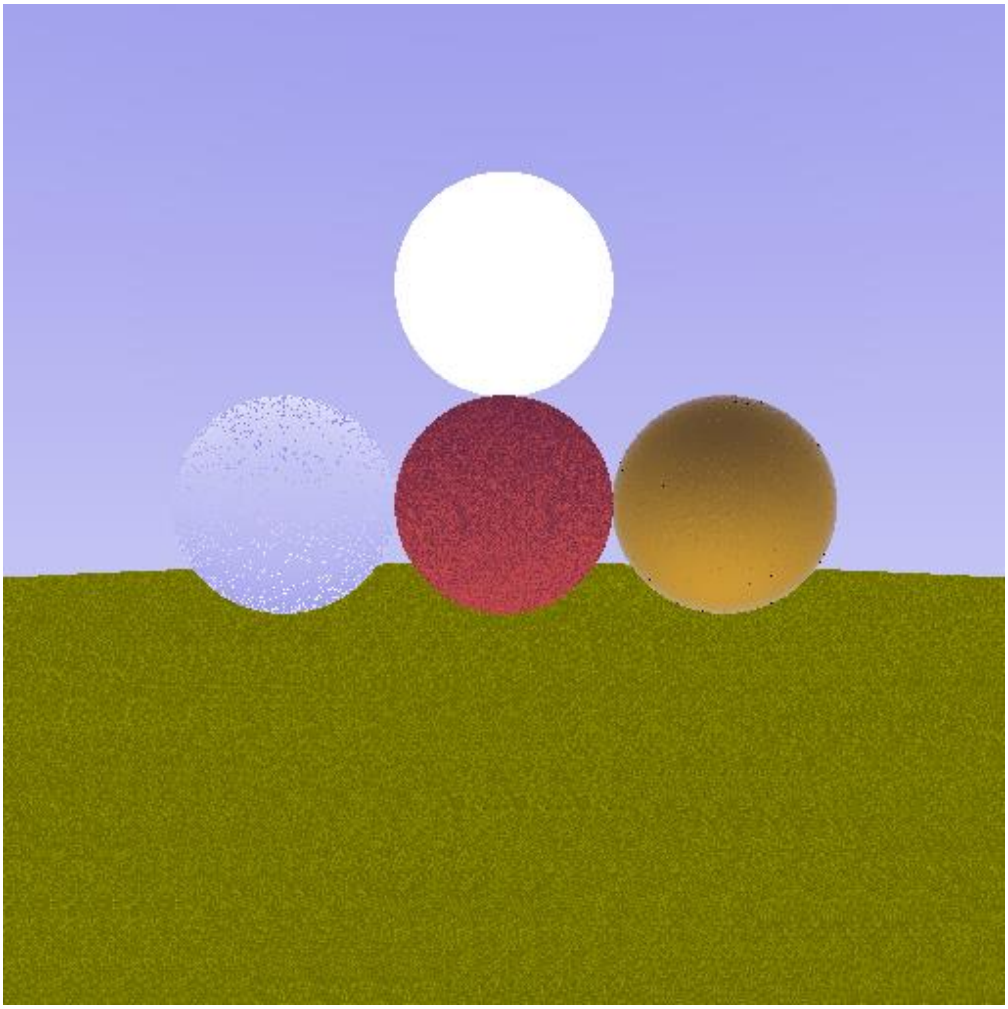


Fig4. Materials_512x512_spp1

2. Indirect Lighting / integrate.h, integrate.cpp

다음 구현 순서는 Antialiasing이지만, Indirect Lighting을 먼저 구현하는 것이 Antialiasing의 효과를 체감하기 좋을 것 같아 Indirect Lighting을 먼저 구현하였다. 해당 기능은 성능적 측면에서 효율적이진 않지만 간단하게 재귀를 통해 구현하였다.

앞서 설명했던 integrate() 함수에서 ray_depth 매개 변수를 추가하고 sampleBackgroundLight()를 통해 배경광만 받던 기존 상황에서 scatter() 함수로부터 생성된 out_ray를 integrate() 함수의 ray 매개 변수에 전달하여 재귀를 실행하도록 하여 구현하였다. ray_depth가 0일시 해당 광선에 대한 기여도는 0으로 처리하기 위해 0 벡터를 그대로 반환하도록 하였다. (주석 처리된 부분은 Direct Light Sampling 부분이다.)

```
glm::vec3 Integrator::integrate(const ne::Ray& ray,
    std::shared_ptr<ne::Scene> scene, int ray_depth, std::vector<ne::RendablePointer> &dls_objects) {
    glm::vec3 L{ 0.0f };
    ne::Ray r{ ray };
    ne::Intersection hit;

    if (ray_depth <= 0)
        return L;

    // if ray hit object in scene, apply shading
    if (scene->rayIntersect(r, hit)) {
        RendablePointer object = hit.object;
        MaterialPointer material = hit.material;

        // avoid double counting
        //if (std::find(dls_objects.begin(), dls_objects.end(), object) == dls_objects.end())
            L += material->emitted();

        ne::Ray out_ray;
        if (hit.material->scatter(r, hit, out_ray)) {

            // Direct Light Sampling
            //dls_objects.clear();
            //glm::vec3 d_L = scene->sampleDirectLight(r, hit, dls_objects);
            //if (d_L != glm::vec3(0.0f))
            //    L += d_L;

            const float pdf = material->pdf(r, hit, out_ray, true);
            if (pdf < 1e-6f)
                return L;
            const glm::vec3 albedo = material->attenuation(r, hit, out_ray, true);
            //L += albedo * scene->sampleBackgroundLight(out_ray.dir) / pdf;
            L += albedo * this->integrate(out_ray, scene, ray_depth - 1, dls_objects) / pdf;
        }
    }
```

결과는 다음과 같다. ray_depth는 15로 설정하였다. (이후 모든 결과도 15로 설정한다.)

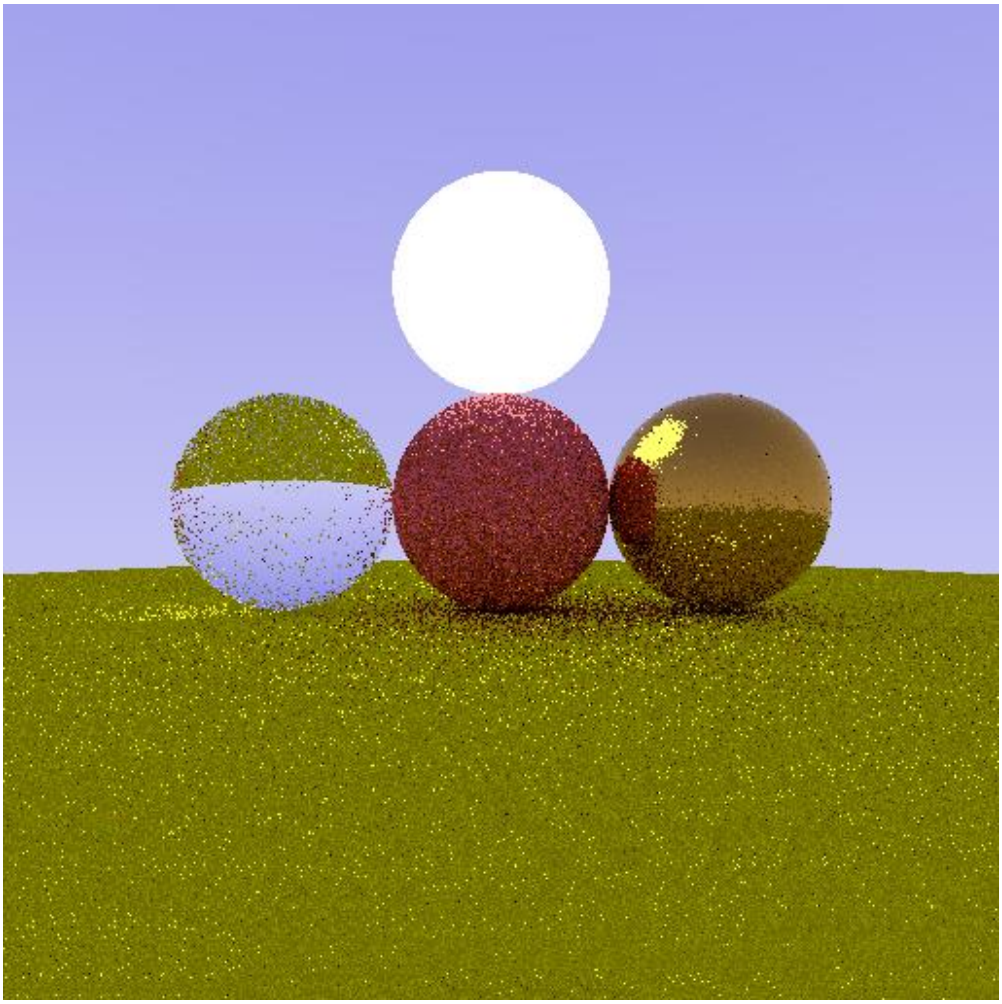
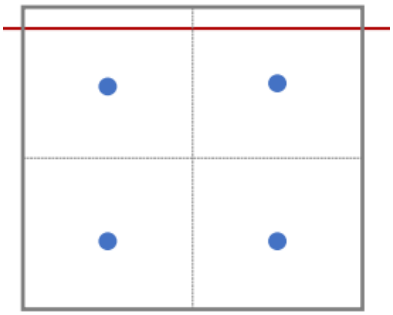


Fig5. IndirectLighting_512x512_spp1

3. Antialiasing / main.cpp

Antialiasing의 경우 가장 기본적인 Regular Grid Sampling으로 구현하였다. 아래의 그림과 같이 픽셀마다 spp개의 격자를 생성하고 격자의 중심마다 ray를 sampling하는 방식이다.



이를 구현하기 위한 방식은 다음과 같다.

1. spp를 가장 가까운 약수쌍으로 분해한다. (spp가 6x6과 같은 제곱수가 아닌 4x8과 같은 수일 경우를 위한 기능)

2. 각각의 약수에 대해 픽셀 for loop 안에서 두 개의 nested for loop를 추가로 실행

3. for loop 내부에서 u, v를 아래와 같이 계산하여 camera.sample() 함수를 실행

float u = float(index.x * ray_x(약수 1) + i + 0.5f) / float(canvas.width() * ray_x);

float v = float(index.y * ray_y(약수 2) + j + 0.5f) / float(canvas.height() * ray_y);

다음은 이를 코드로 표현한 결과이다.

```
// ########## antialiasing ##########
// decide grid size of each pixels.
int sample = static_cast<int>(std::sqrt(spp));
for (sample; sample >= 1; sample--) {
    if (spp % sample == 0)
        break;
}
int ray_x = spp / sample;
int ray_y = sample;
// ########## antialiasing ##########

// build rendering task graph
for (auto &tile : tiles) {
    tf::Task taskTileRender = tf.emplace([&]() {
        // Iterate pixels in tile
        for (auto& index : tile) {
            glm::vec3 color{ 0.0f };

            // construct direct light sampling objects vector
            std::vector<ne::RendablePointer> dls_objects;

            for (int i = 0; i < ray_x; i++) {
                for (int j = 0; j < ray_y; j++) {
                    float u = float(index.x * ray_x + i + 0.5f) / float(canvas.width() * ray_x);
                    float v = float(index.y * ray_y + j + 0.5f) / float(canvas.height() * ray_y);

                    // construct ray and clear direct light sampling objects vector
                    ne::Ray r = camera.sample(u, v);
                    dls_objects.clear();

                    // compute color of ray sample and then add to pixel
                    color += Li.integrate(r, scene, 15, dls_objects);
                }
            }

            color = color / float(spp);
            color = glm::clamp(color, 0.0f, 1.0f);
```

Antialiasing 구현 결과는 다음과 같다. Spp는 1024로 설정하였다.

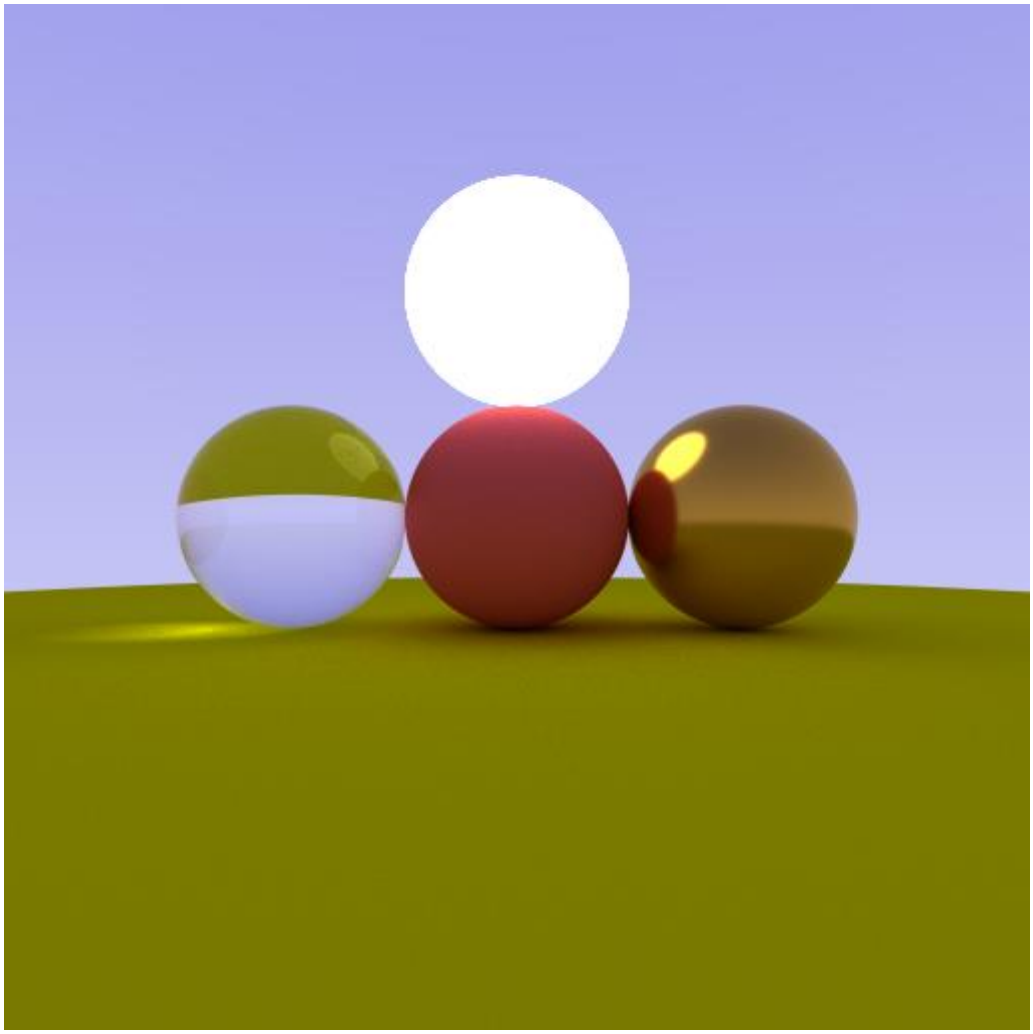


Fig6. Antialiasing_512x512_spp1024

4. Direct Light Sampling / scene.cpp, integrate.cpp

Direct Light Sampling의 경우 광원에 대한 Direct Ray(Shadow Ray)를 추가적으로 sampling하여 적은 spp에서 Indirect Light가 광원을 hit하지 못했을 때에 생기는 noise를 줄이기 위한 기능이다. Direct Light Sampling을 구현하기 위한 방식으로 충돌 지점마다 scene의 모든 광원에 대해 각각 한 개의 Direct Light Ray를 Sampling하는 방식을 채택했다.

Direct Light Sampling의 경우 과제에서는 Sphere Area Light만을 고려하기 때문에 Cone Sampling 방식을 사용한다. Cone Sampling의 경우 한 지점에서 구에 대해 보이는 부분의 입체각의 범위에 대해서만 Sampling하는 방식으로 그림으로 표현할 경우 다음과 같다.

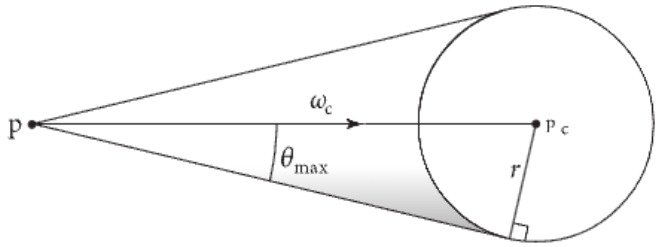


Figure 6.5: To sample points on a spherical light source, we can uniformly sample within the cone of directions around a central vector ω_c with an angular spread of up to θ_{max}. Trigonometry can be used to derive the value of $\sin\theta_{max}$, $r/|p_c - p|$.

PBR 4ed 6.2.4

해당 샘플링의 구현 방식은 교차 지점(hit point)과 구형 광원의 중심 사이의 방향을 기준으로 최대 입사각을 계산하고, 이 범위 내에서 난수를 이용해 임의의 방향을 샘플링하는 방식이다. 샘플링된 방향은 로컬 좌표계에서 정의된 뒤, 월드 좌표계로 변환되어 최종적으로 Shadow Ray의 방향으로 사용된다. 이를 코드로 표현하면 다음과 같다.

```
glm::vec3 Scene::sampleDirectLight(const ne::Ray& r_in, ne::Intersection &hit) const {
    glm::vec3 L{ 0.0f };
    ne::Ray shadow_ray;
    ne::Intersection shadow_hit;

    for (const auto li : lights_) {
        auto l_sphere = std::dynamic_pointer_cast<ne::Sphere>(li);
        glm::vec3 lightDir = l_sphere->center_ - hit.p;
        float d2 = glm::dot(lightDir, lightDir);
        float d = std::sqrt(d2);

        float sinThetaMax = l_sphere->radius_ / d;
        float cosThetaMax = std::sqrt(glm::max(0.0f, 1.0f - sinThetaMax * sinThetaMax));
        float oneMinusCosThetaMax = 1.0f - cosThetaMax;

        float u1 = glm::linearRand(0.0f, 1.0f);
        float u2 = glm::linearRand(0.0f, 1.0f);

        float cosTheta = 1.0f - u1 * (1.0f - cosThetaMax);
        float sinTheta = std::sqrt(std::max(0.0f, 1.0f - cosTheta * cosTheta));
        float phi = 2.0f * glm::pi<float>() * u2;

        glm::vec3 dirLocal(
            sinTheta * cos(phi),
            sinTheta * sin(phi),
            cosTheta);

        glm::vec3 wc = glm::normalize(lightDir);
        glm::vec3 wi = glm::transpose(changeToLocal(wc)) * dirLocal;

        shadow_ray.dir = glm::normalize(wi);
        shadow_ray.o = hit.p + li->eps_ * shadow_ray.dir;
```

이후 Shadow Ray와 선택된 광원에 대해 rayIntersect() 함수를 실행시켜서 t 값을 광원과 hit 지점 사이의 거리로 제한해놓고 scene의 모든 오브젝트에 대해 Intersect를 검사한다. (선택된 광원은 제외된다.) Intersect가 없을 경우 hit 물체의 Material의 attenuation() 함수 값과 Cone Sampling의 PDF 값을 곱해주고 전체 광량인 L에 추가해준다. PDF의 경우 물체의 PDF 값을 그대로 사용하지 않는 이유는 Direct Light Sampling은 BSDF 분포를 기반으로 진행한 것이 아닌 Cone Sampling을 기반으로 새로운 Sampling 방식을 채택했기 때문에 동일한 PDF를 사용해서는 안되기 때문이다. Cone Sampling의 PDF는 다음과 같다.

$$\frac{1}{2\pi(1-\cos\theta_{\max})}$$

sampleDirectLight() 함수의 나머지 영역은 위의 내용을 코드로 표현한 것이다.

```
        if (!li->rayIntersect(shadow_ray, shadow_hit)) continue;

        bool foundIntersection = false;
        for (const auto o : objects_) {
            if (o == li)
                continue;
            if (o->rayIntersect(shadow_ray, shadow_hit)) {
                foundIntersection = true;
                break;
            }
        }
        if (!foundIntersection) {
            const glm::vec3 albedo = hit.material->attenuation(r_in, hit, shadow_ray, false);
            if (albedo == glm::vec3(0.0f)) continue;

            float pdf = 1.0f / (2.0f * glm::pi<float>() * oneMinusCosThetaMax);
            L += albedo * li->material_->emitted() / pdf;
            dls_objects.push_back(li);
        }
    }

    return L;
}
```

Direct Light Sampling에서 추가적으로 고려해야 하는 것은 광원의 빛에 대한 이중 기여 문제이다. 만약 충돌 지점에서 Direct Light Sampling을 진행하였고, 해당 Sample이 광원에 적중한다면 충돌 지점에는 직접광에 대한 기여가 생긴다. 그리고 이후에 Indirect Light Sampling을 진행하고 scatter된 ray가 또 다시 동일한 광원에 적중한다면 광원의 빛에 대한 이중 기여가 발생한다. 이는 과도한 빛 합산을 유발하여, 결국은 noise를 DLS를 진행함에도 줄여주지 못하는 문제가 발생한다. 따라서 만약 DLS가 기여를 했다면, Indirect Light에 대해서는 DLS에 기여한 광원에 적중했을 때 해당 기여를 0으로 판단하는 기능을 추가한다.

```
// if ray hit object in scene, apply shading
if (scene->rayIntersect(r, hit)) {
    RendablePointer object = hit.object;
    MaterialPointer material = hit.material;

    // avoid double counting
    if (std::find(dls_objects.begin(), dls_objects.end(), object) == dls_objects.end())
        L += material->emitted();
```

결과는 다음과 같다.

Direct Light Sampling 적용 전 (spp 32)

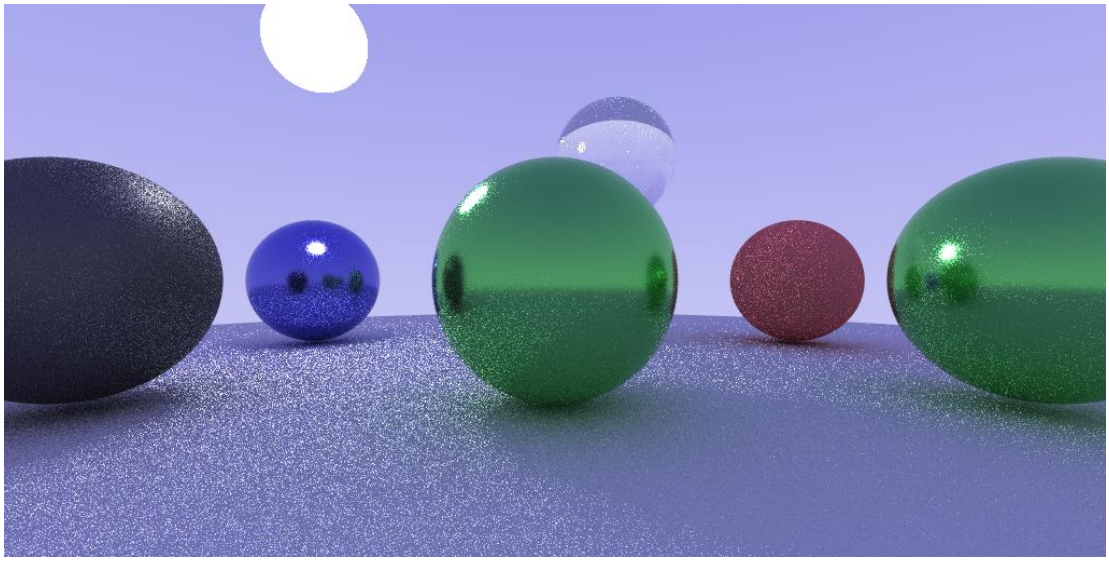


Fig7. BeforeDLS_1024x512_spp32

적용 후 (spp 32)

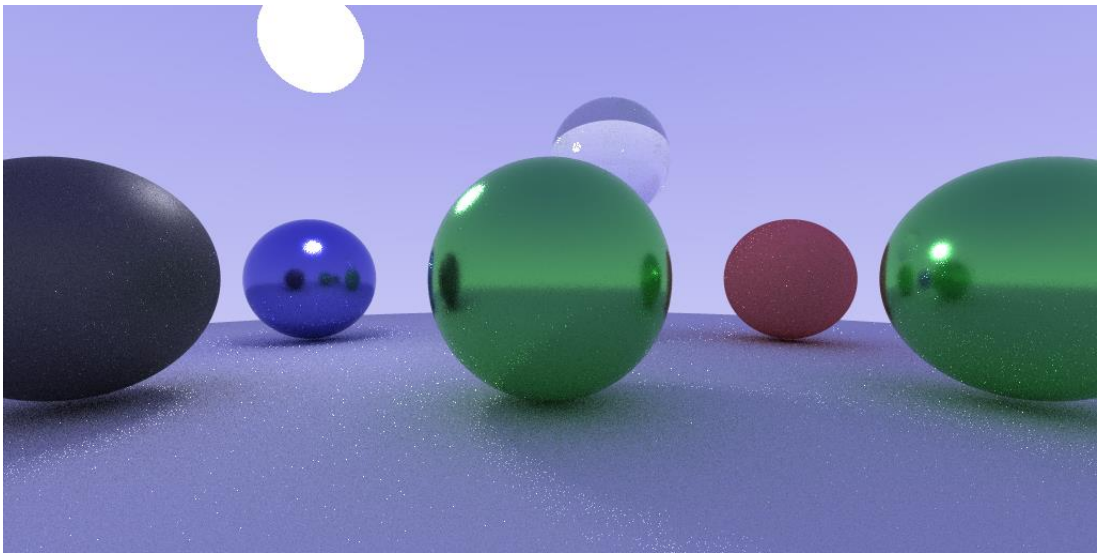


Fig8. AfterDLS_1024x512_spp32

32라는 적은 spp에서도 noise가 눈에 띄게 감소한 모습을 확인할 수 있다.

추가적으로 DLS를 적용하지 않고 Indirect Light만으로 spp를 늘렸을 때(1024)와 비교하기 위해 렌더링한 이미지인데, with DLS와 큰 차이가 없다는 것을 확인할 수 있었고 따라서 DLS의 샘플링 방식과 PDF 설정이 적절했음을 확인할 수 있었다.

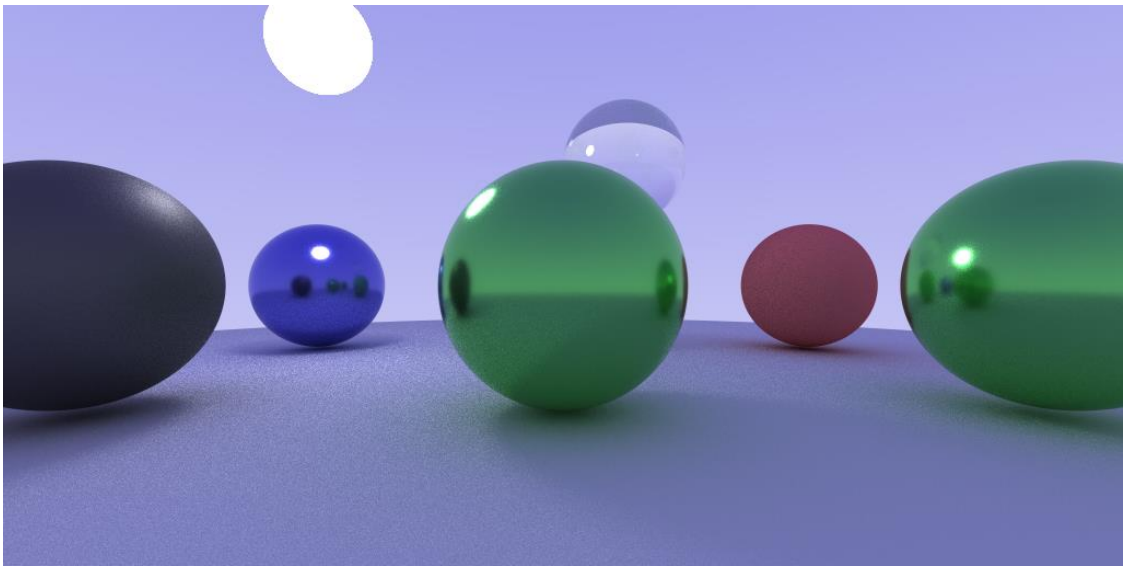


Fig9 NoDLS_1024x512_spp1024